

Simulation - Network-based

2026-04-30

Define or generate the population $\mathcal{I}$ and interference structure M .

We generate a simple random network interference structure. The population is divided into K disconnected network components. Within each component, the edges are generated as a random regular graph. Interference occurs only through immediate network neighbors.

Let $K = 1000$ denote the number of disconnected network components. Each component has 10 individuals, and each individual has 4 network neighbors. For individual i , the interference set $\mathcal{N}_i$ consists of the nodes adjacent to i in the network, with cardinality $|\mathcal{N}_i|$ equal to the node degree.

After generating the edge list, we use the package function `build_network()` to create the network object. We also independently generate a binary individual-level covariate

$$L_i \sim \text{Bernoulli}(0.4), \quad i \in \mathcal{I}.$$

```
set.seed(999)

K <- 1000
component_size <- 10
degree <- 4

network_edges <- do.call(rbind, lapply(seq_len(K), function(k) {
  ids <- ((k - 1) * component_size + 1):(k * component_size)
  g <- igraph::sample_k_regular(
    no.of.nodes = component_size,
    k = degree,
    directed = FALSE,
    multiple = FALSE
  )
  e <- igraph::as_edgelist(g, names = FALSE)
  data.frame(x1 = ids[e[, 1]], x2 = ids[e[, 2]])
}))

net.network <- build_network(
  x1 = network_edges$x1,
  x2 = network_edges$x2
)

net.network$data$network_id <- ceiling(net.network$data$subject_id / component_size)
net.network$data$L_i <- rbinom(nrow(net.network$data), 1, 0.4)
```

Simulate $Y(a_i, a_{\mathcal{N}_i})$.

The outcome-generating mechanism corresponds to a binary outcome and is specified using the aggregated network-neighbor exposure mapping $\bar{a}_{\mathcal{N}_i}$:

$$Y(a_i, a_{\mathcal{N}_i}) = Y(a_i, \bar{a}_{\mathcal{N}_i}) = \text{logit}^{-1}(0.5 - 0.5a_i - 0.5\bar{a}_{\mathcal{N}_i} + 0.25a_i\bar{a}_{\mathcal{N}_i} + 0.25L_i).$$

Coefficients are chosen to induce both direct and spillover effects and to allow for interaction between individual and peer exposure; their specific values are not substantively interpreted.

Simulate A_i and $\bar{A}_{\mathcal{N}_i}$.

Given the treatment assignment mechanism, we generate a single realization of the treatment vector $\mathbf{A} = (A_1, \dots, A_n)$. For each individual i , the aggregated exposure $\bar{A}_{\mathcal{N}_i}$ is defined as the mean treatment status among individuals in i 's network-neighbor interference set, excluding i themselves.

We assume that treatment is assigned according to the following individual-level model:

$$A_i \sim \text{Bernoulli}\{\text{logit}^{-1}(-0.5 + L_i)\}.$$

```
sim.network <- simulate_network_outcome(
  net.network,
  outcome_intercept = 0.5,
  beta_a = -0.5,
  beta_abar = -0.5,
  beta_a_abar = 0.25,
  outcome_covars = "L_i",
  outcome_coefs = 0.25,
  propensity_intercept = -0.5,
  treatment_covars = "L_i",
  treatment_coefs = 1
)

PO <- sim.network[[1]]
dat.network <- sim.network[[2]]
```

`PO` is a list of matrices, with each matrix corresponding to an individual i and encoding that individual's potential outcomes under different treatment and interference scenarios. Within each matrix, the rows index the individual's own treatment status, with the first row representing individual i not treated ($A_i = 0$) and the second row representing individual i treated ($A_i = 1$). The columns index the level of exposure to interference, where column k corresponds to the case in which exactly k individuals in i 's network-neighbor interference set are treated, for $k = 0, 1, \dots, N_i$, with N_i denoting the size of the interference set for individual i .

`dat.network` is observed outcome using consistency $Y_{\text{obs},i} = PO_i[A_i + 1, A_{\mathcal{N}_i} + 1]$.

```
PO[[1]]
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    1    0    0    1
## [2,]    0    0    0    0    1
```

```
PO[[2]]
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    1    1    1    1
## [2,]    0    0    0    1    1
```

```
head(dat.network)
```

```
##   subject_id id network_id L_i interference_set_size Ai treated_neighbors
## 1          1  1          1  0                   4  1              1
## 2          2  2          1  0                   4  0              0
## 3          3  3          1  1                   4  1              1
## 4          4  4          1  1                   4  0              3
## 5          5  5          1  0                   4  1              2
## 6          6  6          1  1                   4  1              0
##   Abar_Ni Y_obs
## 1    0.25    0
## 2    0.00    1
## 3    0.25    1
## 4    0.75    0
## 5    0.50    0
## 6    0.00    0
```

Calculate true causal effects

The object `P0` stores individual-level potential outcomes under network interference. For each individual i , `P0[[i]]` is a $2 \times (N_i + 1)$ matrix, where rows correspond to the individual's own treatment status ($A_i = 0$ or $A_i = 1$), and columns correspond to the number of treated individuals in i 's network-neighbor interference set ($k = 0, \dots, N_i$).

To compute true causal estimands under a Bernoulli allocation policy with treatment probability α , we average potential outcomes over the distribution of the number of treated individuals in the network-neighbor interference set, induced by independently assigning treatment to each neighbor with probability α . For fixed (a, α) , the individual-level average potential outcome is defined as

$$\bar{Y}_i(a_i = a, \alpha) = \sum_{k=0}^{|\mathcal{N}_i|} \text{PO}_i[a + 1, k] \binom{|\mathcal{N}_i|}{k} \alpha^k (1 - \alpha)^{|\mathcal{N}_i| - k}.$$

The package function below implements these quantities directly from `sim.network`.

```
true_val <- simulate_network_true_effect(
  sim.network,
  allocations = c(0.25, 0.5, 0.75)
)
```

```
true_val
```

```
## $allocations
## [1] 0.25 0.50 0.75
##
## $DE
##   effect alpha true_effect
## 1    DE  0.25 -0.11141641
## 2    DE  0.50 -0.09586250
## 3    DE  0.75 -0.07964141
##
## $IE
##   effect alpha1 alpha0 true_effect
```

```
## 1    IE    0.25    0.50  0.02946836
## 2    IE    0.25    0.75  0.06083125
## 3    IE    0.50    0.75  0.03136289
##
## $TE
##   effect alpha1 alpha0 true_effect
## 1     TE    0.25    0.50 -0.08194805
## 2     TE    0.25    0.75 -0.05058516
## 3     TE    0.50    0.75 -0.06449961
##
## $OE
##   effect alpha1 alpha0 true_effect
## 1     OE    0.25    0.50  0.04954551
## 2     OE    0.25    0.75  0.09270820
## 3     OE    0.50    0.75  0.04316270
##
## attr(,"class")
## [1] "network_true_effect"
```

Estimate the spillover effect using inference.

The group variable in `inference()` is `network_id`, which identifies the disconnected network component. There are no edges between different `network_id` groups.

```
model <- inference(
  Y_obs | Ai ~ L_i | network_id,
  allocations = c(0.25, 0.5),
  data = dat.network,
  model_method = "glm",
  model_options = list(family = binomial(link = "logit"))
)
```

```
## Warning: the 'findbars' function has moved to the reformulas package. Please update your imports, or
## This warning is displayed once per session.
```

```
model$estimates
```

##	alpha1	trt1	alpha2	trt2	marginal	effect_type	effect	estimate
## 1	0.25	0	NA	NA	FALSE	outcome	outcome	0.590615639
## 2	0.50	0	NA	NA	FALSE	outcome	outcome	0.572999940
## 3	0.25	1	NA	NA	FALSE	outcome	outcome	0.468214024
## 4	0.50	1	NA	NA	FALSE	outcome	outcome	0.477923589
## 5	0.25	NA	NA	NA	TRUE	outcome	outcome	0.560015235
## 6	0.50	NA	NA	NA	TRUE	outcome	outcome	0.525461764
## 7	0.25	1	0.25	0	FALSE	contrast	direct	-0.122401614
## 8	0.50	1	0.50	0	FALSE	contrast	direct	-0.095076351
## 9	0.25	0	0.25	1	FALSE	contrast	direct	0.122401614
## 10	0.50	0	0.50	1	FALSE	contrast	direct	0.095076351
## 11	0.25	0	0.25	0	FALSE	contrast	indirect	0.000000000
## 12	0.50	0	0.25	0	FALSE	contrast	indirect	-0.017615699
## 13	0.25	1	0.25	1	FALSE	contrast	indirect	0.000000000
## 14	0.50	1	0.25	1	FALSE	contrast	indirect	0.009709565

```

## 15  0.25  0  0.50  0  FALSE  contrast indirect  0.017615699
## 16  0.50  0  0.50  0  FALSE  contrast indirect  0.000000000
## 17  0.25  1  0.50  1  FALSE  contrast indirect -0.009709565
## 18  0.50  1  0.50  1  FALSE  contrast indirect  0.000000000
## 19  0.25  1  0.25  0  FALSE  contrast   total -0.122401614
## 20  0.50  1  0.25  0  FALSE  contrast   total -0.112692049
## 21  0.25  1  0.50  0  FALSE  contrast   total -0.104785915
## 22  0.50  1  0.50  0  FALSE  contrast   total -0.095076351
## 23  0.25  0  0.25  1  FALSE  contrast   total  0.122401614
## 24  0.50  0  0.25  1  FALSE  contrast   total  0.104785915
## 25  0.25  0  0.50  1  FALSE  contrast   total  0.112692049
## 26  0.50  0  0.50  1  FALSE  contrast   total  0.095076351
## 27  0.25  NA  0.25  NA  TRUE   contrast overall  0.000000000
## 28  0.50  NA  0.25  NA  TRUE   contrast overall -0.034553471
## 29  0.25  NA  0.50  NA  TRUE   contrast overall  0.034553471
## 30  0.50  NA  0.50  NA  TRUE   contrast overall  0.000000000
##      std.error      conf.low  conf.high
## 1  0.059587660  0.473825972  0.707405305
## 2  0.012406656  0.548683341  0.597316538
## 3  0.024731526  0.419741124  0.516686924
## 4  0.011353177  0.455671771  0.500175407
## 5  0.046077676  0.469704649  0.650325821
## 6  0.008965814  0.507889092  0.543034437
## 7  0.060784943 -0.241537913 -0.003265315
## 8  0.015623578 -0.125698001 -0.064454700
## 9  0.060784943  0.003265315  0.241537913
## 10 0.015623578  0.064454700  0.125698001
## 11 0.000000000  0.000000000  0.000000000
## 12 0.056681582 -0.128709559  0.093478161
## 13 0.000000000  0.000000000  0.000000000
## 14 0.025381772 -0.040037795  0.059456925
## 15 0.056681582 -0.093478161  0.128709559
## 16 0.000000000  0.000000000  0.000000000
## 17 0.025381772 -0.059456925  0.040037795
## 18 0.000000000  0.000000000  0.000000000
## 19 0.060784943 -0.241537913 -0.003265315
## 20 0.062091607 -0.234389362  0.009005263
## 21 0.025563929 -0.154890296 -0.054681535
## 22 0.015623578 -0.125698001 -0.064454700
## 23 0.060784943  0.003265315  0.241537913
## 24 0.025563929  0.054681535  0.154890296
## 25 0.062091607 -0.009005263  0.234389362
## 26 0.015623578  0.064454700  0.125698001
## 27 0.000000000  0.000000000  0.000000000
## 28 0.045376000 -0.123488796  0.054381854
## 29 0.045376000 -0.054381854  0.123488796
## 30 0.000000000  0.000000000  0.000000000

```

Simulation framework to evaluate and compare true versus estimated spillover (indirect) effects.

We conduct a simulation study to evaluate the performance of estimators for spillover effects under network interference. In each of 500 iterations, we use a fixed random network structure, regenerate the covariate,

treatment, potential outcomes, and observed outcome, compute the true indirect effect under two allocation strategies (0.25 vs. 0.5) using the known data-generating mechanism, and estimate the corresponding indirect effect using `inference()`.

```
set.seed(999)

K <- 100
component_size <- 10
degree <- 4

network_edges <- do.call(rbind, lapply(seq_len(K), function(k) {
  ids <- ((k - 1) * component_size + 1):(k * component_size)
  g <- igraph::sample_k_regular(
    no.of.nodes = component_size,
    k = degree,
    directed = FALSE,
    multiple = FALSE
  )
  e <- igraph::as_edgelist(g, names = FALSE)
  data.frame(x1 = ids[e[, 1]], x2 = ids[e[, 2]])
}))

inference.network <- build_network(
  x1 = network_edges$x1,
  x2 = network_edges$x2
)

inference.network$data$network_id <- ceiling(
  inference.network$data$subject_id / component_size
)

iter <- 500
sim.results <- data.frame(
  true_val = numeric(iter),
  estimate = numeric(iter),
  std.error = numeric(iter),
  conf.low = numeric(iter),
  conf.high = numeric(iter)
)

for (i in 1:iter) {
  net.i <- inference.network
  net.i$data$L_i <- rbinom(nrow(net.i$data), 1, 0.4)

  sim.network <- simulate_network_outcome(
    net.i,
    outcome_intercept = 0.5,
    beta_a = -0.5,
    beta_abar = -0.5,
    beta_a_abar = 0.25,
    outcome_covars = "L_i",
    outcome_coefs = 0.25,
    propensity_intercept = -0.5,
    treatment_covars = "L_i",
  )
}
```

```

    treatment_coefs = 1
  )

  dat.network <- sim.network[[2]]
  true_val <- simulate_network_true_effect(sim.network, allocations = c(0.25, 0.5))

  model <- interference(
    Y_obs | Ai ~ L_i | network_id,
    allocations = c(0.25, 0.5),
    data = dat.network,
    model_method = "glm",
    model_options = list(family = binomial(link = "logit"))
  )

  estimate_val <- subset(
    model$estimates,
    alpha1 == 0.25 & alpha2 == 0.5 &
      effect == "indirect" & trt1 == 0 & trt2 == 0
  )

  true_ie <- subset(
    true_val$IE,
    alpha1 == 0.25 & alpha0 == 0.5
  )$true_effect

  sim.results[i, ] <- c(
    true_ie,
    estimate_val$estimate,
    estimate_val$std.error,
    estimate_val$conf.low,
    estimate_val$conf.high
  )
}

avg.bias <- mean(sim.results$true_val - sim.results$estimate)
ECP <- mean(between(sim.results$true_val, sim.results$conf.low, sim.results$conf.high))

avg.bias

## [1] 0.000825964

ECP

## [1] 0.812

```